Image Enhancement, Restoration & Histogram Equalization

https://www.dynamsoft.com/blog/insights/image-processing/image-processing-101-image-enhancement/

https://www.youtube.com/watch?v=0udRjwIuvjA

https://www.youtube.com/watch?v=vafrnbwGONc

Why do we need image enhancement

- To **improve visual quality**, **reveal hidden details**, and **prepare images for computer analysis**
- **Fixing poor lighting:** Brightens dark photos or tones down bright spots so hidden features become easy to see
- **Removing noise:** Cleans up grainy or fuzzy pictures caused by bad cameras or low light
- **Sharpening edges:** Makes blurry lines and shapes clear for better viewing
- **Boosting contrast:** Separates an object from its background when colors or shades look too similar
- **Aiding technology:** Helps computers, medical tools, and security systems recognize faces, maps, or diseases accurately

Spatial Domain

- Spatial domain methods **directly manipulate the pixel values of an image.** This approach is intuitive and often effective, as it works directly with the raw pixel data.
- **Operations are performed directly on the image matrix**, where the value of a pixel is altered based on its own value and/or the value of neighbouring pixels within a defined area

Point Operations (Brightness, Contrast, Thresholding)

- These operations modify individual pixels values independently of their location or neighbouring pixels. They use a transformation function to map input pixel values to new output values.
- Common types of Point Operation include:
- ***Brightness adjustment***
- ***Contrast stretching***
- ***Thresholding***

Spatial Filtering (Smoothing, Sharpening, Edge Detection)

- These techniques modify a pixel's value based on the values of its neighbours within a defined window or kernel. This approach is commonly used for tasks such as smoothing sharpening, and noise reduction.
- Common types of Spatial Filtering include:
- ***Smoothing (deblurring)***
- ***Sharpening***
- ***Edge detection***

Frequency Domain

- Frequency domain methods operate on the ***Fourier Transform of an image rather than directly on its pixels***.
- The Fourier Transform converts an image from its spatial representation to a frequency representation, where image ***information like brightness variations and edges are represented by different frequencies***.

Continuation…..

- For instance, filtering techniques are applied to the transformed image, and then an inverse Fourier Transform is performed to convert the image back to the spatial domain.

Continuation…..

- This approach is particularly powerful for tasks like:
- **Noise Reductions**: By selectively attenuating high-frequency components that often correspond to noise.
- **Image sharpening**: By enhancing high-frequency components related to edges and details.
- **Homomorphic Filtering**: A specialized technique used to correct non-uniform illumination and simultaneously enhance contrast by separating the illumination and reflectance component of an image.

Contrast Enhancement / Stretching

- When an image looks dull or lacks variation between light and dark areas, it ***often means the pixel values are clustered in a narrow range.***
- Contrast adjustment redistributes or stretches these values so that features stand out distinctly.

An image has pixel values ranging from **40 to 180**. Stretch the intensity range to **0–255**. Find the new value of pixel intensity **100**.

Given:

- $r = 100$
- $r_{\min} = 40$
- $r_{\max} = 180$
- $L = 256$

Solution

Formula:

$$s = \frac{(r - r_{\min})(L - 1)}{r_{\max} - r_{\min}}$$

$$s = \frac{(100 - 40) \times 255}{180 - 40}$$

$$= \frac{60 \times 255}{140} = 109.3$$

$$\boxed{s \approx 109}$$

Histogram Equalization

- This technique is especially ***useful for images taken in poor lighting***.
- Histogram equalization spreads pixel intensities more evenly across the available range, helping reveal details that were previously hidden in shadows or highlights.

Gray level	0	1	2	3
Frequency	2	3	4	7

- Find the equalized value of intensity **2**.

- Total pixels N=2+3+4+7=16
- CDF for intensity 2 2+3+4=9
- Formula

$$s = \left\lfloor \frac{CDF}{N}(L-1) \right\rfloor$$

$$s = \left\lfloor \frac{9}{16} \times 3 \right\rfloor$$

$$= \lfloor 1.6875 \rfloor = 1$$

$$\boxed{1}$$

- Where L = 4

The probability of a normal variable $P(z > 1.8) = 0.0359$. A random variable X is normally distributed with mean $\mu = 50$ and standard deviation $\sigma = 10$. Compute $P(X < 68)$.

Given Mean, μ=50, Standard deviation, σ=10,

Find P(X<68)

Step 1: Convert X to the standard normal variable Z

$$Z = \frac{X - \mu}{\sigma}$$

For $X = 68$,

$$Z = \frac{68 - 50}{10} = \frac{18}{10} = 1.8$$

Thus,

$$P(X < 68) = P(Z < 1.8)$$

Step 2: Use the given probability

The problem states:

$$P(Z > 1.8) = 0.0359$$

Since the total probability is 1,

$$P(Z < 1.8) = 1 - P(Z > 1.8)$$

$$= 1 - 0.0359$$

$$= 0.9641$$

Final Answer

$$\boxed{P(X < 68) = 0.9641}$$

Continuation…..

Enhancement Techniques:

- Convert RGB to Gray Scale Image
- Contrast Stretching
  a. for Gray Scale Images
  b. for RGB Images
- Histogram Equalization
  a. for Gray Scale Images
  b. for RGB Images



[1]**Reference:** Photo Enhancement (Graphhenne Graphics)



IMAGE ENHANCEMENT

Histogram Equalization

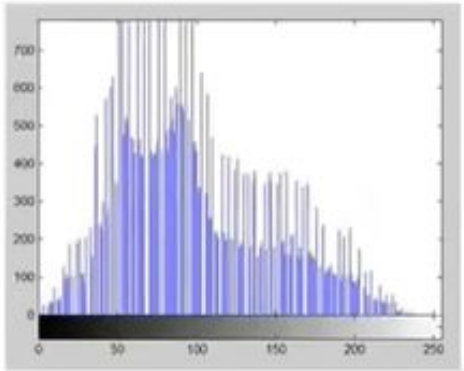
Old image



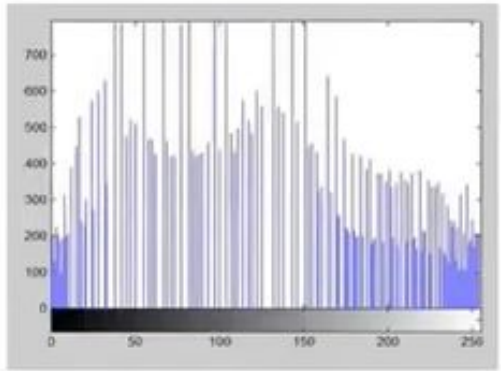
New Image



Old Histogram



New Histogram



[7]**Reference:** Histogram Equalization (tutorialspoint.com)

EQUATION:

$$CDF(x) = \sum_{k=-\infty}^{x} P(k)$$

$$P(k) = (\mathrm{L} - 1)\, CDF$$

where:
CDF = Cumulative Density Function
P(k) = Number of Pixels for a given gray value
L - 1 = highest gray value

[8]**Reference:** Histogram Equalization formula (www.mygreatlearning.com)

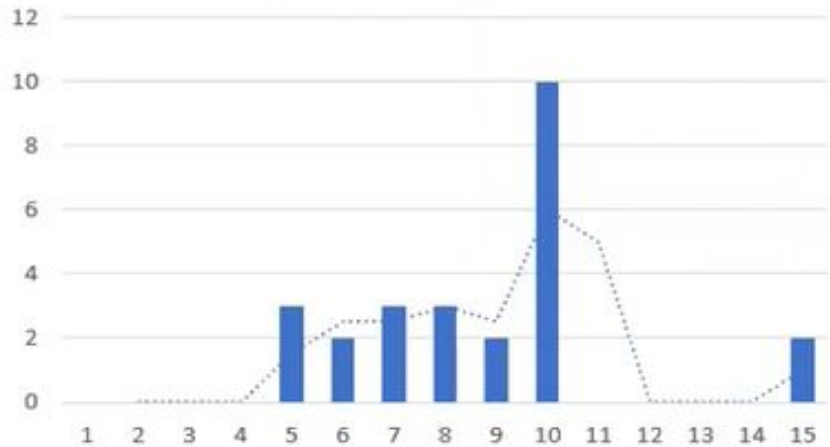


IMAGE ENHANCEMENT

Let's put this in practice!

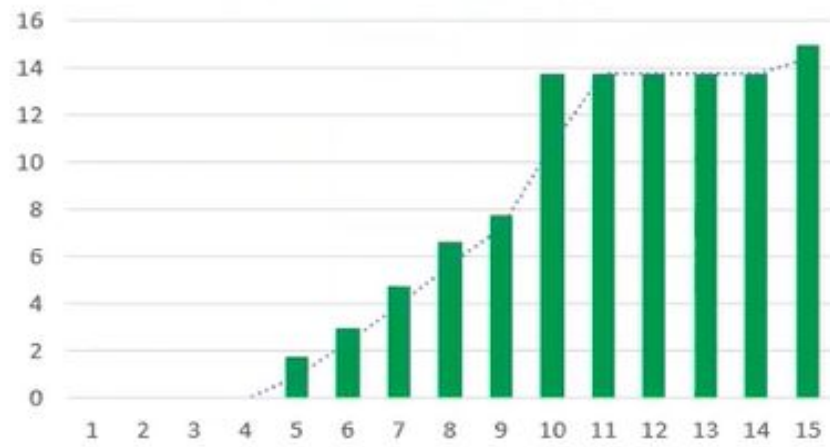
Original Image Pixels				
8	5	6	10	8
5	7	8	13	10
7	10	10	15	14
10	9	9	5	12
15	10	14	7	6

Original Frequency Histogram



Output Image Pixels				
7	2	3	11	7
2	5	7	13	11
5	11	11	15	14
11	8	8	2	12
15	11	14	5	3

Output Frequency Histogram



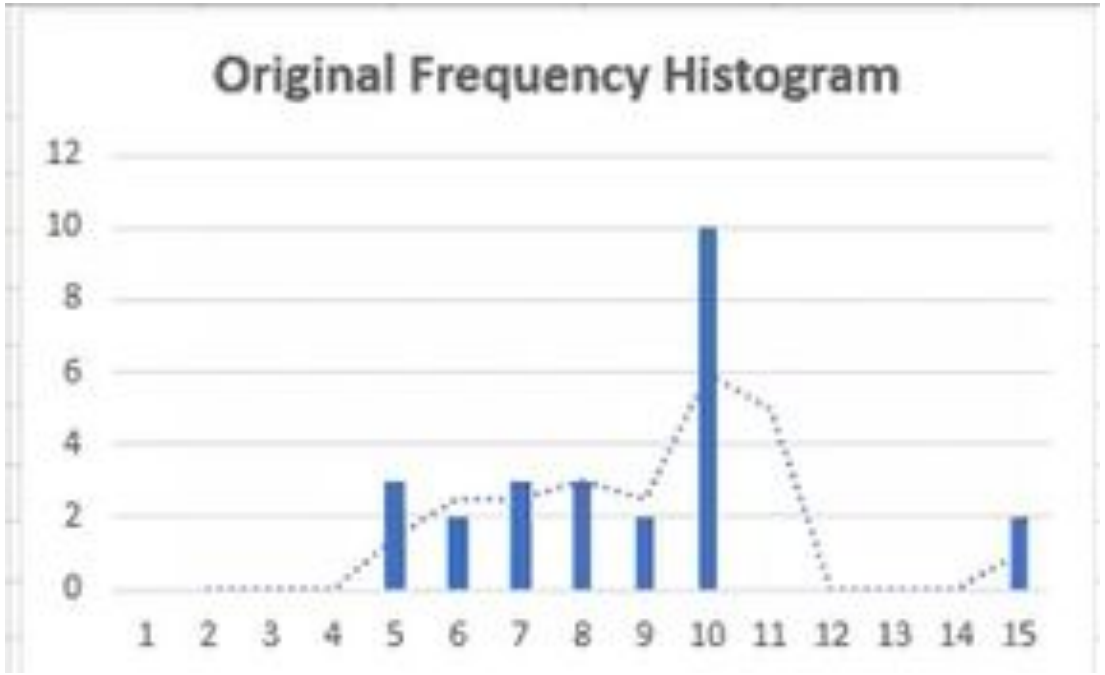
Original Image Pixels				
8	5	6	10	8
5	7	8	10	10
7	10	10	15	10
10	9	9	5	10
15	10	10	7	6

Highest Gray Value = 15		
2^4 = 16	16 bits	[0 15]

Output Image Pixels				
7	2	3	14	7
2	5	7	14	14
5	14	14	15	14
14	8	8	2	14
15	14	14	5	3

Gray Level	# of Input Pixels	PDF #pixels/su	CDF cum. sum of pixels	CDF * N	Output [CDF(x)]
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0
5	3	0.12	0.12	1.8	2
6	2	0.08	0.2	3	3
7	3	0.12	0.32	4.8	5
8	3	0.12	0.44	6.6	7
9	2	0.08	0.52	7.8	8
10	10	0.4	0.92	13.8	14
11	0	0	0.92	13.8	14
12	0	0	0.92	13.8	14
13	0	0	0.92	13.8	14
14	0	0	0.92	13.8	14
15	2	0.08	1	15	15
N	25				

Continuation…..



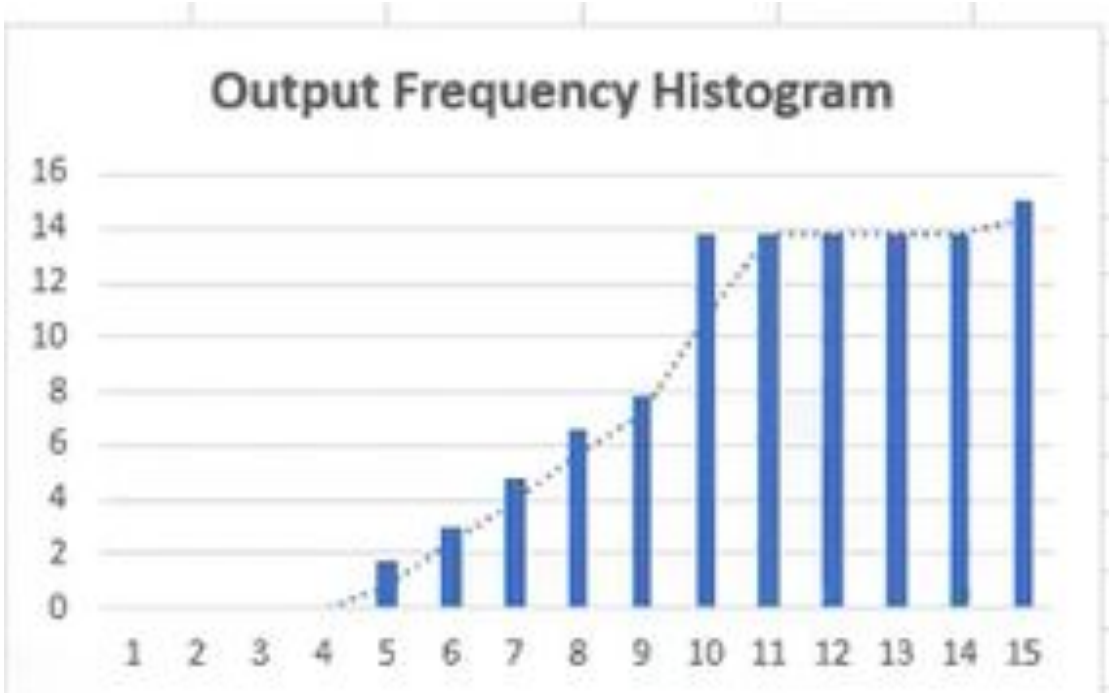


IMAGE RESTORATION

Restoration Techniques:

- Mean or Averaging Filter
  a. for Gray Scale Images
  b. for RGB Images



[9]Reference: Photo-restoration (www.ittogo.co.uk)



IMAGE RESTORATION

Mean or Averaging Filter



[10]Reference: Ironman Images

EQUATION:

$$I_f(x,y) = \frac{1}{m*n}\sum \hat{I}(x,y)$$

where:
$\hat{I}(x,y)$ = corrupted image due to noise
$I_f(x,y)$ = filtered or restored image
m x n = kernel to be used in mean filtering

[11]Reference: Mean Filter (https://homepages.inf.ed.ac.uk)



IMAGE RESTORATION

Mean or Averaging Filter

Input (i,j)

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	90	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$* \quad \frac{1}{9} \quad x$

Kernel

1	1	1
1	1	1
1	1	1

Output (i,j)

0	10	20	30	30	30	20	10
0	20	30	50	50	60	40	20
0	30	50	80	80	90	60	30
0	30	80	80	80	90	60	30
10	30	60	90	90	90	60	30
10	30	40	60	60	50	40	20
10	20	20	30	30	30	20	10
10	10						



IMAGE RESTORATION

Mean or Averaging Filter for Gray Scale Images

MATLAB Code:

Editor - C:\Users\ADMIN\Desktop\SampleCode.m

SampleCode.m

```
%% Image Restoration for Gray Scale Image
% Mean Filter or Averaging Filter
im = imread("PiliNuts.jpg");
im = im2gray(im);
N = imnoise(im,"salt & pepper",0.03);
H = fspecial("average",3)
imRstr = imfilter(N,H);
subplot(2,2,1), imshow(im), title("Original Image");
subplot(2,2,2), imshow(N), title("Noisy Image");
subplot(2,2,3), imshow(imRstr), title("Restored Image");
```



Noisy Image

Restored Image



Original I



Noisy Image



Restored Image





IMAGE RESTORATION

Mean or Averaging Filter for RGB Images

MATLAB Code:

Editor - C:\Users\ADMIN\Desktop\SampleCode.m

SampleCode.m

```
57      %% Image Restoration for Colored Image
58      %Mean Filter or Averaging filter
59 -    im = imread("PiliNuts.jpg");
60 -    N = imnoise(im,"gaussian");
61 -    H = fspecial("average",3);
62 -    imRstr = imfilter(N,H);
63 -    subplot(2,2,1), imshow(im), title("Original Image");
64 -    subplot(2,2,2), imshow(N), title("Noisy Image");
65 -    subplot(2,2,3), imshow(imRstr), title("Restored Image");
```



Noisy Image

Restored Image



Original Image



Noisy Image



Restored





OTHER IMAGE FILTERS

Gaussian Blur,Sharpen Kernel, Edge Detection

Before



*

1/16	1/8	1/16
1/8	1/4	1/8
1/16	1/8	1/16

=

After



Gaussian Blur

Before



*

-1	-1	-1
-1	8	-1
-1	-1	-1

=

After



Edge Detection

Before



*

0	-1	0
-1	5	-1
0	-1	0

=

After



Sharpen Kernel

[12]*Reference Video:*
Sharpening Spatial filters in digital image processing with examples

MATLAB Code

```
% Image Restoration using Mean (Average) Filter
clc;
clear;
close all;
% Step 1: Read the input image
I = imread('/MATLAB Drive/13.jpg');
% Convert to grayscale if the image is RGB
if size(I,3) == 3
I = rgb2gray(I);
end
% Display original image
figure;
subplot(1,2,1);
imshow(I);
title('Original Noisy Image');
% Step 2: Create a 3x3 Average Filter
H = fspecial('average',[3 3]);
% Step 3: Apply Mean Filter
Restored = imfilter(I,H,'replicate');
% Display restored image
subplot(1,2,2);
imshow(Restored);
title('Restored Image using Mean Filter');
```



THANK YOU